

# LE REFLUX · le test d'un an, avec le moteur de l'agent

2026-09-18 · demandé par Mongazi : « fais le test sur 1 an avec un capital de 10 dollars sur chacun, et prends les notes pour le deep learning ».

## Le verdict, d'abord

| critère de Mongazi                               | NAS100                                     | EUR/USD                                      |
|--------------------------------------------------|--------------------------------------------|----------------------------------------------|
| plus de 50 % des trades à 2 R                    | <b>non : 30,8 %</b>                        | <b>non : 25,4 %</b>                          |
| résultat par trade                               | <b>-0,103 R</b>                            | <b>-0,370 R</b>                              |
| 10 \$ avec le plafond de levier de l'agent (x30) | <b>0 trade</b> (il faut ~80 \$)            | <b>0 trade</b> (il faut ~39 \$)              |
| 10 \$ sans plafond de levier                     | 2 trades, 2 pertes, <b>10 \$ → 8,98 \$</b> | 10 trades, 10 pertes, <b>10 \$ → 6,62 \$</b> |
| taille sans contrainte (100 000 \$)              | <b>-98,9 %</b> en un an                    | <b>-100 %</b> en un an                       |

**La méthode LE REFLUX, telle qu'un courtier peut l'exécuter, perd de l'argent sur les deux marchés.** Aucun réglage du plan de risque ne change ça : une espérance négative perd plus vite quand on risque plus, et elle perd moins vite quand on risque moins, mais elle perd.

## Pourquoi la recherche disait l'inverse

Le simulateur de recherche ( `banc._simuler_ordres` ) traite les ordres limites **dans l'ordre où ils ont été posés**. La stratégie pose un ordre à chaque minute, valable une heure : plusieurs attendent donc en même temps. Quand le prix plonge à travers plusieurs ordres d'achat, le simulateur donne le trade **au plus ancien qui finit par être servi**, c'est-à-dire au plus bas. Il sait donc que le prix ira jusque-là, et il ignore l'ordre du haut, servi le premier, qui continue de baisser et touche son stop. Chez un courtier, c'est **l'ordre touché le premier** qui entre.

Même année (2025-09-17 → 2026-09-17), mêmes prix et coûts Deriv, mêmes filtres :

| NAS100                    | trades | 2 R atteints | par trade       |
|---------------------------|--------|--------------|-----------------|
| recherche, sans filtre    | 7 174  | 40,0 %       | +0,186 R        |
| <b>agent, sans filtre</b> | 8 011  | 31,0 %       | <b>-0,090 R</b> |
| recherche, filtre 5 %     | 340    | 67,3 %       | +0,987 R        |
| <b>agent, filtre 5 %</b>  | 1 457  | 30,8 %       | <b>-0,103 R</b> |

| EUR/USD                  | trades | 2 R atteints | par trade       |
|--------------------------|--------|--------------|-----------------|
| recherche, sans filtre   | 7 153  | 36,7 %       | -0,035 R        |
| recherche, filtre 5 %    | 366    | 60,4 %       | +0,681 R        |
| <b>agent, filtre 5 %</b> | 1 045  | 25,4 %       | <b>-0,370 R</b> |

Le filtre, lui, a appris **sur les trades du simulateur** : il a appris à reconnaître les situations où le simulateur trichait avec le plus de profit. Chez le courtier, ces situations n'existent pas.

**Le moteur de l'agent est contrôlé contre le simulateur** ( `python -m trading.recherche._qc_moteur` ) : avec des ordres valables une minute, un seul ordre attend à la fois et les deux doivent être d'accord. Ils donnent les mêmes trades au dix-millième de R près, et les 208 écarts observés sur 60 000 minutes commencent **tous** par le même second défaut du simulateur, lui aussi du côté optimiste : il laisse un nouvel ordre être servi dans la minute même où le trade précédent s'arrête, sans savoir lequel des deux est arrivé en premier.

## Les variantes honnêtes, toutes perdantes (NAS100, même année)

Si l'avantage venait des entrées profondes, une règle qui les cherche sans lire l'avenir devrait le retrouver. Elle ne le retrouve pas ( `python -m trading.recherche.rejeu_variantes` ) :

| variante causale                        | sans filtre               | filtre 5 %                |
|-----------------------------------------|---------------------------|---------------------------|
| d'origine : premier touché, repli 0,5 R | 8 011 · 31,0 % · -0,090 R | 1 457 · 30,8 % · -0,103 R |
| n'armer que l'ordre le plus profond     | 3 829 · 31,1 % · -0,080 R | 25 · 24,0 % · -0,310 R    |
| premier touché, repli 1,0 R             | 5 512 · 30,3 % · -0,107 R | 161 · 29,8 % · -0,142 R   |
| premier touché, repli 1,5 R             | 3 140 · 30,8 % · -0,092 R | 13 · 23,1 % · -0,339 R    |

Le point mort d'un objectif à 2 R est vers 34 % une fois les coûts payés : **aucune variante ne l'atteint**. Ces variantes ont été choisies après avoir vu la première perdre : elles comptent au registre comme des essais, pas comme des preuves.

## Le capital minimum (la question de Mongazi)

|                                            | NAS100             | EUR/USD                            |
|--------------------------------------------|--------------------|------------------------------------|
| lot minimum                                | 0,1                | 0,01                               |
| stop médian de la stratégie                | 24 points d'indice | 3,7 pips                           |
| risque du lot minimum                      | ~2,40 \$           | ~0,37 \$                           |
| capital pour que ce risque tienne dans 6 % | <b>~40 \$</b>      | <b>~6 \$</b> (12 \$ au palier 3 %) |
| capital sous le plafond de levier x30      | <b>~80 \$</b>      | <b>~39 \$</b>                      |
| levier médian à 6 % de risque              | x29                | x73                                |

➔ **Augmenter le risque pour faire entrer un petit capital ne répare rien ici** : la méthode perd par trade. Le seul effet serait de vider le compte en moins de trades.

## Comment ce test a été fait

- **Le moteur de décision est celui de l'agent** ( `live/moteur_scalp.py` ), utilisé par le rejeu ET par `live/scalpeur.py` : un ordre candidat par minute ( `candidates_v2.rabais` , les formules de la recherche), les ordres de l'autre sens abandonnés au retournement, un seul ordre armé à la fois, celui que le prix touchera le premier, et seulement si le filtre approuve la minute qui vient de se fermer. Remplissage : le prix doit traverser la limite d'un spread entier.
- **Filtres sans regard vers l'avenir** : un modèle par trimestre, appris sur les seuls trades clos avant lui (Dukascopy, tout l'historique), seuil = quantile des probabilités hors échantillon des quatre trimestres précédents (NAS100 : 0,627 · 0,625 · 0,625 · 0,622 ; EUR/USD : 0,569 · 0,563 · 0,565 · 0,564).
- **Prix et coûts Deriv**, minute par minute. **Vrais lots** ( `noyau/risque.dimensionner` ) et **échelle 6-4-3** ( `noyau/profils.risque_courant` ) : le code de l'agent.
- **Caractéristiques calculées par morceaux** ( `caracteristiques.construire_aux_barres` ) et contrôlées contre le calcul complet ( `_qc_parite.py` ) : identiques à 2 millièmes près avec 20 000 minutes d'historique. ⚠ Avec les 6 000 minutes que lisait l'agent, une caractéristique s'écartait de 3,7 %.

## Les notes, trade par trade

Chaque trade du rejeu est écrit en entier dans `trading/rapports/recherche/rejeu/*.jsonl` (hors de GitHub) : les 38 caractéristiques de la minute qui a armé l'ordre, la probabilité et le seuil, la limite voulue et le prix obtenu, le coût payé à l'entrée et à la sortie, le plus loin allé pour ( `mfe_R` ) et contre ( `mae_R` ), l'âge de l'ordre, la durée, le motif de sortie, le palier de risque, les lots, le levier et le résultat. L'agent en direct écrit la même fiche dans `trading/rapports/recherche/direct/<marché>.jsonl` , et régénère `trading/SUIVI.md` à chaque trade fermé.

Ce que les notes disent déjà : **l'âge médian d'un ordre servi est d'une minute**. L'agent entre sur les tout premiers replis, et 999 trades NAS100 sur 1 457 finissent au stop, contre 449 à l'objectif. Le meilleur moment médian d'un trade est à +0,85 R : il n'approche même pas les 2 R.

**⚠ Sur l'apprentissage profond** : les notes sont la bonne matière, mais un réseau appris sur quelques milliers de trades apprend le bruit. Le modèle actuel (arbres de décision, 80 000 à 160 000 exemples) est déjà l'outil adapté à cette taille. Ce qui a trompé tout le monde, ce n'est pas le modèle, ce sont **les exemples** : il faut les produire avec le moteur de l'agent, pas avec le simulateur.

## La suite, dans cet ordre

---

1. **Réapprendre le filtre sur les trades de l'agent** (moteur causal), et le rejuger sur l'année. C'est la seule piste qui reste pour cette famille de stratégies ; elle peut échouer.
2. **Corriger le simulateur de recherche** (l'ordre touché le premier, pas de réentrée dans la barre de sortie) et **rejouer tout le registre** : d'autres « découvertes » peuvent reposer sur le même défaut.
3. **Aucun argent réel, aucune démo « pour voir »** tant qu'une version causale n'est pas positive sur une période qu'elle n'a jamais vue.

Données : une journée EUR/USD Dukascopy est corrompue (2024-10-10, 617 bougies dont le plus haut passe sous la clôture) : `_qc_sans_fin` la signe en rouge, à retélécharger.